

Calloway OS

Omni-Core Telemetry Station

Project Report

Author	Logan Calloway
Platform	XIAO ESP32-C6
Version	v2.0
Date	July 2026

“A centralized, glanceable engineering dashboard designed to monitor a workspace and provide real-time environmental and network data from a single, always-on embedded device – now paired with a live web dashboard for historical trends.”

Abstract

This report documents the design, development, and architecture of the Omni-Core Telemetry Station, a custom desktop IoT dashboard built on the XIAO ESP32-C6 microcontroller running Calloway OS.

The goal of this project was straightforward: build a device that never crashes, always knows where it is, and fills a screen with real, useful data pulled from both the internet and local sensors. Most IoT projects work until they don't. A dropped WiFi connection, a bad API response, or a stalled HTTP request can bring everything down silently. This project was designed from the ground up to handle those situations gracefully and recover on its own.

The device pulls time from NTP servers, weather from OpenWeatherMap, and reads live environmental data from a TSL2591 light sensor, an SCD40 CO2/temperature/humidity sensor, and an SEN54 particulate matter and VOC sensor. Location and timezone are detected automatically via IP geolocation, which means the device works correctly whether it is sitting on a desk at home or plugged into a power outlet somewhere else entirely. Everything updates itself with no configuration needed after first setup.

Version 2.0 adds a second way to look at the same data: a live web dashboard, served directly from the device itself at `omnicore.local`, with a full 24-hour history of every reading and a set of charts anyone can pull up from a phone or laptop on the same network. The dashboard and the physical screen are both driven by the same underlying sensor readings, so neither one is ever the “real” source of truth over the other; they are just two windows into the same data.

The software architecture enforces a strict separation between data fetching, display rendering, and system coordination. This makes the codebase easy to extend as new sensors arrive and keeps individual functions small and honest about what they do.

Keywords: ESP32-C6, IoT, embedded systems, telemetry, NTP, OpenWeatherMap, ST7796, capacitive touch, TSL2591, SCD40, SEN54, particulate matter, air quality, C++, Arduino framework

Contents

1	Introduction	4
2	Project Goals	4
3	Hardware Design	5
3.1	Platform	5
3.2	Display	5
3.3	Touch Input	5
3.4	Sensors	6
3.5	Backlight Control	6
3.6	PCB Design	6
3.7	3D Enclosure Design	10
4	Understanding the Air Quality Data	15
4.1	CO2 – Carbon Dioxide	15
4.2	Particulate Matter – PM1, PM2.5, PM4, and PM10	16
4.3	VOC Index	16
4.4	Why CO2 and PM2.5 Specifically Drive the Status Indicator	17
5	Software Architecture	17
5.1	Three-Layer Design	17
5.2	Struct-Scoped State	17
5.3	Non-Blocking Loop	18
5.4	Canvas Buffering	18
5.5	Color Palette System	18
5.6	Settings Menu	18
5.7	Settings Persistence	18
6	The Web Dashboard	19
6.1	Motivation	19
6.2	Live Data Endpoint	19
6.3	History Ring Buffer	19
6.4	Charts and Range Selection	19
6.5	Air Quality Status Indicator	20
6.6	A Note on Multi-File Builds	20
7	Reliability Design	20
8	Development History	21
9	Current Status and Future Work	23
9.1	What’s Working Now	23
9.2	Next Steps	23

10 Conclusion	24
11 Appendix: Pinout and Wiring Reference	25
11.1 XIAO ESP32-C6 Pin Assignments	25
11.2 Shared I2C Bus	25
11.3 Power and Decoupling	25
11.4 Datasheet Links	26
11.5 Bill of Materials	26
11.6 Figures	27

Introduction

Most IoT devices are built to work in one place, on one network, with one fixed configuration. They tend to be brittle in ways that only show up after you have been using them for a while. A router change breaks the WiFi, a timezone shift breaks the clock, or a brief power outage leaves the device in a state it can never recover from on its own.

The Omni-Core Telemetry Station was built to be different. The core question driving every design decision was: *what happens when something goes wrong, and how does the device get itself back to a good state without any help?*

That question shaped everything. WiFi credentials are stored in non-volatile memory and the device reconnects automatically if the signal drops. Location and timezone are detected from the network rather than hardcoded, so moving the device to a different location does not break anything. A hardware watchdog timer resets the chip if the main loop ever stalls. HTTP requests have timeouts so a slow API server cannot freeze the display. A 6-hour reconnect limit triggers a clean restart if WiFi never comes back.

The display side was designed with the same thinking. Off-screen canvas buffers are used for every region of the screen that updates frequently, so each region redraws atomically with no visible flicker. Only the parts of the display that actually changed get redrawn on each tick. The color palette is defined in a single place and everything on screen, including the settings menu, brightness slider, and reset confirmation dialog, responds to a Night Mode toggle that shifts the entire UI to a red palette, which protects the user's night vision and circadian rhythm during late-night sessions.

Version 2.0 extends the project in a direction the original design did not originally plan for: history. The physical screen has always been, by design, a snapshot of right now. It has never tried to show a trend, because a 480x320 panel simply does not have room for one alongside everything else on it. The web dashboard exists to answer the question the screen never could: what has this room actually been doing for the last hour, the last six hours, the last day?

This report covers the hardware design, software architecture, key engineering decisions, and the development history of the project from initial prototype to the current version. The full source code is available at <https://github.com/logancalloway/calloway-os>.

Project Goals

The Omni-Core was built around a few core requirements that stayed constant throughout development.

- **Reliability above everything.** The device should run unattended for months without intervention. Every failure mode should have a recovery path.
- **Location independence.** The device should work correctly anywhere with an internet connection. No manual reconfiguration required.

- **Real data only.** Nothing on the display should show a placeholder or a zero when real data is not available yet. The display waits for valid data before drawing anything.
- **Glanceability.** The physical dashboard should communicate everything the user needs at a glance, including time, date, weather, local environmental readings, air quality status, and WiFi status, without requiring any interaction.
- **Expandability.** The architecture should make it easy to add new sensors without restructuring existing code.
- **Trend visibility.** Beyond a single glanceable moment, the device should be able to answer “what has this room been like recently,” without needing a separate computer, database, or cloud service to do it.

Hardware Design

Platform

The XIAO ESP32-C6 was chosen as the platform for several reasons. It runs at 160MHz with 512KB of RAM and 4MB of flash, which is significantly more headroom than the project currently needs and leaves room for future expansion. The built-in WiFi and Bluetooth stack handles all network communication without any additional hardware. The RISC-V core architecture is increasingly relevant in professional embedded work, making this a good platform to develop skills on.

Display

The dashboard moved from an earlier 2.8-inch resistive display to a 3.5-inch ST7796 IPS TFT running at 480x320, driven over SPI by the Arduino_GFX library. The larger panel gave enough room to add a full particulate matter row and an overall air quality status line without crowding the clock, date, or weather. Six GFXcanvas16 off-screen buffers now cover the clock, date, local sensor readout, particulate row, air quality row, and status row, each one drawing completely into RAM before a single SPI transfer pushes the finished region to the screen.

Touch Input

The original hardware-button interface was replaced with an FT6236 capacitive touch controller, sharing the same I2C bus as the environmental sensors. Tapping the dashboard opens a full settings menu with scrollable items, a live-dragging brightness slider, and a two-step confirmation screen before a WiFi reset is allowed to happen. Touch position is only ever trusted at the start of a touch, since the FT6236 reports an unreliable position at the moment a finger lifts off the glass.

One quirk worth documenting here: the FT6236’s own `begin()` call reports failure on this specific board even when touch is working correctly. Rather than trust that return value,

the firmware logs that initialization was attempted and treats an actual touch on the glass as the real verification.

Sensors

Sensor	Purpose	Status
TSL2591	Ambient light (lux)	Active
SCD40	CO2, temperature, humidity	Active
SEN54	Particulate matter (PM1/2.5/4/10), VOC index	Active

All three sensors share one I2C bus on pins D4 (SDA) and D5 (SCL) at 400kHz. The TSL2591 has a dynamic range of up to 88,000 lux, making it suitable for both dark room and bright daylight readings, and its reading drives both the vertical lux bar on the dashboard and the Auto Dim brightness feature. The SCD40 provides local temperature, humidity, and true CO2 concentration via nondispersive infrared sensing, updating the top-left of the dashboard. The SEN54 replaced an earlier air quality sensor and adds real particulate matter readings across four size classes plus a VOC index, discussed in detail in Section 4.

Backlight Control

The backlight is driven by direct PWM to the display’s LED pin. The Brightness menu item cycles through four fixed steps (25%, 50%, 75%, 100%), and the Auto Dim setting maps the TSL2591’s live lux reading directly onto the PWM duty cycle, so the screen dims and brightens automatically as the room’s ambient light changes.

PCB Design

The board was designed in EasyEDA Standard and fabricated and assembled by JLCPCB.

The PCB was designed to be small and practical rather than densely packed, prioritizing signal integrity and thermal isolation over board area. The XIAO ESP32-C6 mounts directly to the board, with the USB-C port left accessible from the side of the finished enclosure for reprogramming and power without needing to open the case.

The SCD40’s footprint includes an open cutout beneath and around the sensor rather than a solid copper pour. CO2 sensing of this kind depends on accurate local temperature, and a solid ground/power pour right under the sensor would let heat from the microcontroller and regulator conduct into the SCD40 and skew its readings. The cutout keeps that heat path broken and gives the sensor better airflow to read the room’s actual air rather than the board’s.

Dedicated connectors are broken out for the SEN54 and for the display module, rather than soldering either directly to the main board, so either one can be serviced or swapped without touching the rest of the board. A separate PWM line is broken out for backlight control, driven directly from the microcontroller as described in Section 3.5.

Decoupling follows standard practice for a mixed digital/sensor board: bulk 10 μ F capacitors on the board's main supply rails (one each for 3.3V and 5V), 1 μ F capacitors at the SCD40 and TSL2591, a 10 μ F bulk capacitor at the SEN54's supply pins, and 100nF capacitors placed as close as possible to individual power pins to suppress high-frequency switching noise before it can couple into the I2C bus or the sensors reading it. All three sensors and the touch controller share one I2C bus, so keeping that bus electrically quiet mattered more here than it would on a board with only one sensor. Pull-up resistors on the I2C data and clock lines are sized at 4.7k Ω , standard for this bus speed and trace length. The complete pin assignments and I2C address map are collected in Section 11.

Two revisions of this board have been built. The current revision adds an updated footprint and 5V supply for the SEN54, corrected after the first prototype run.

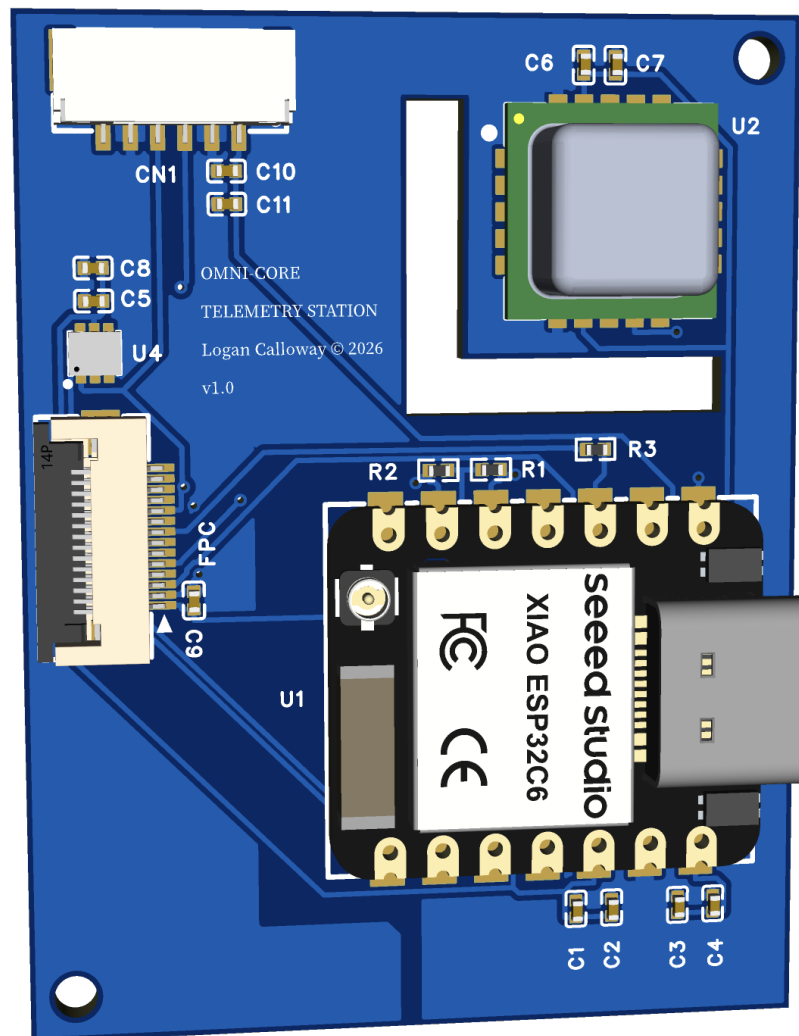


Figure 1: PCB, 3D render, front side.

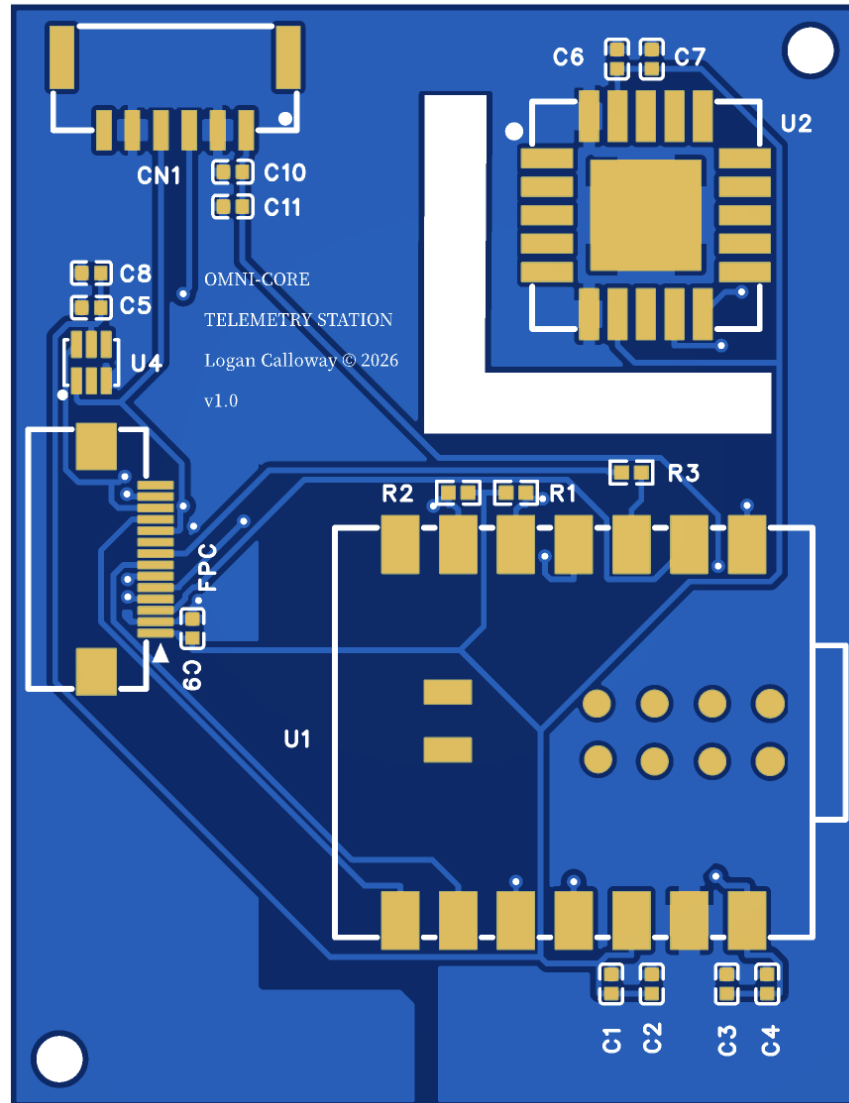


Figure 2: PCB layout, top copper layer.



Figure 3: PCB layout, bottom copper layer.

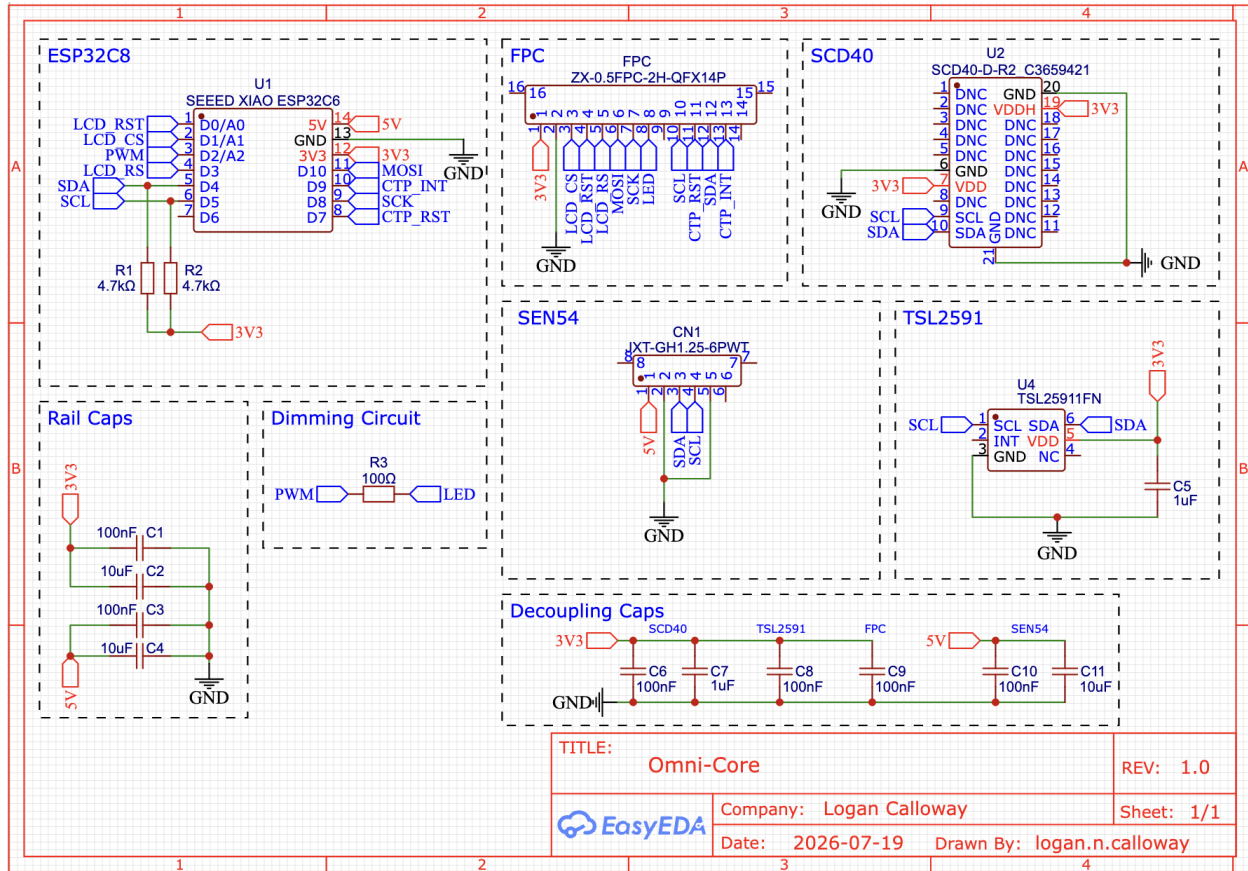


Figure 4: Omni-Core schematic, rev 1.0.

3D Enclosure Design

The enclosure was modeled in SolidWorks and 3D printed.

The enclosure's design constraints came directly from what the electronics actually need physically: airflow to the outside world for the environmental sensors, secure mounting for the PCB and display, and access to the board's connectors without needing screws or tools to open the case.

The back panel uses a hexagonal perforation pattern, open straight through to the outside air. The SCD40's CO₂ reading and the SEN54's particulate reading are only meaningful if the sensors are actually sampling the room's air rather than a sealed pocket of trapped, self-heated air inside the case, so this pattern exists specifically to keep outside air moving through the enclosure. The SEN54 also draws air through its own internal fan, and a separate side vent was added specifically to give that fan a clear, unobstructed intake path, rather than relying on the back panel's general airflow alone.

The PCB mounts inside the main body of the enclosure with the USB-C port aligned to a side cutout, so the board can be powered and reprogrammed without opening the case. A

press-fit cutout on the inside of the enclosure holds the SEN54 firmly in place against the side vent, without needing an adhesive or additional fasteners.

The lid is a separate piece, shaped to fit the 3.5” display exactly and sit flush with the outside face of the enclosure once assembled. Rather than using screws, the lid and body use a snap-fit design, small mating features molded into both halves that click together and hold under normal handling, with no visible screw holes anywhere on the finished enclosure.



Figure 5: Enclosure, assembled, front view. The on-screen UI shown is an AI-generated (ChatGPT) mockup of the dashboard, not a capture from the running device.

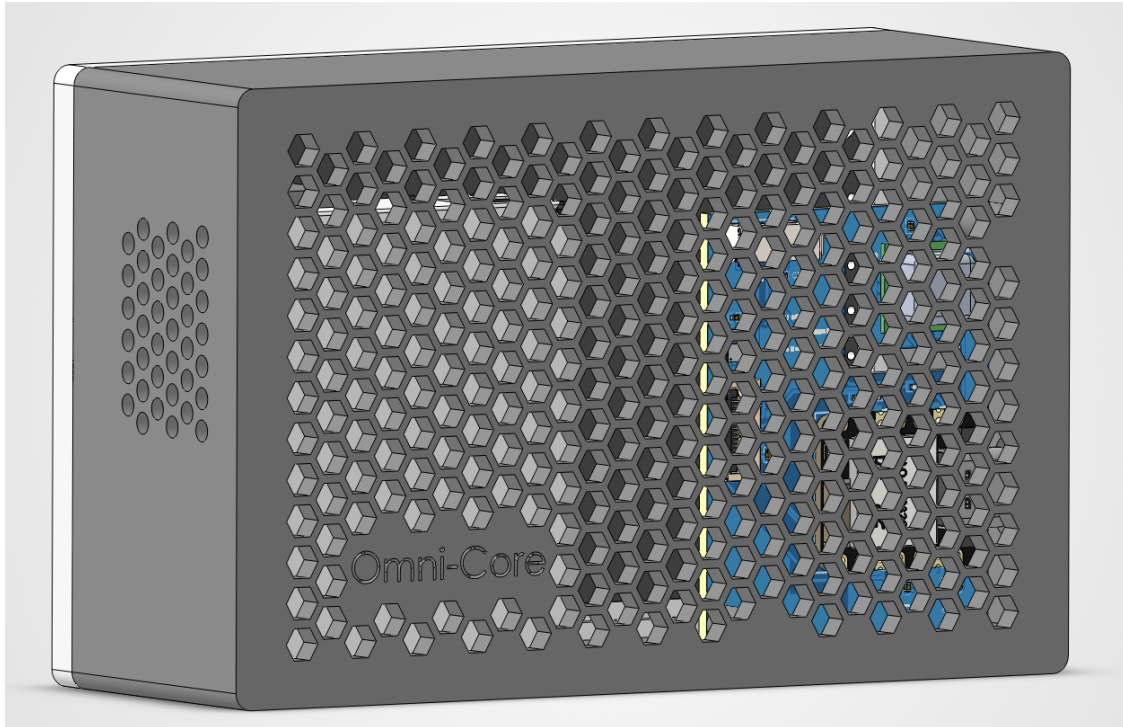


Figure 6: Back panel, hexagonal airflow pattern and side vent.

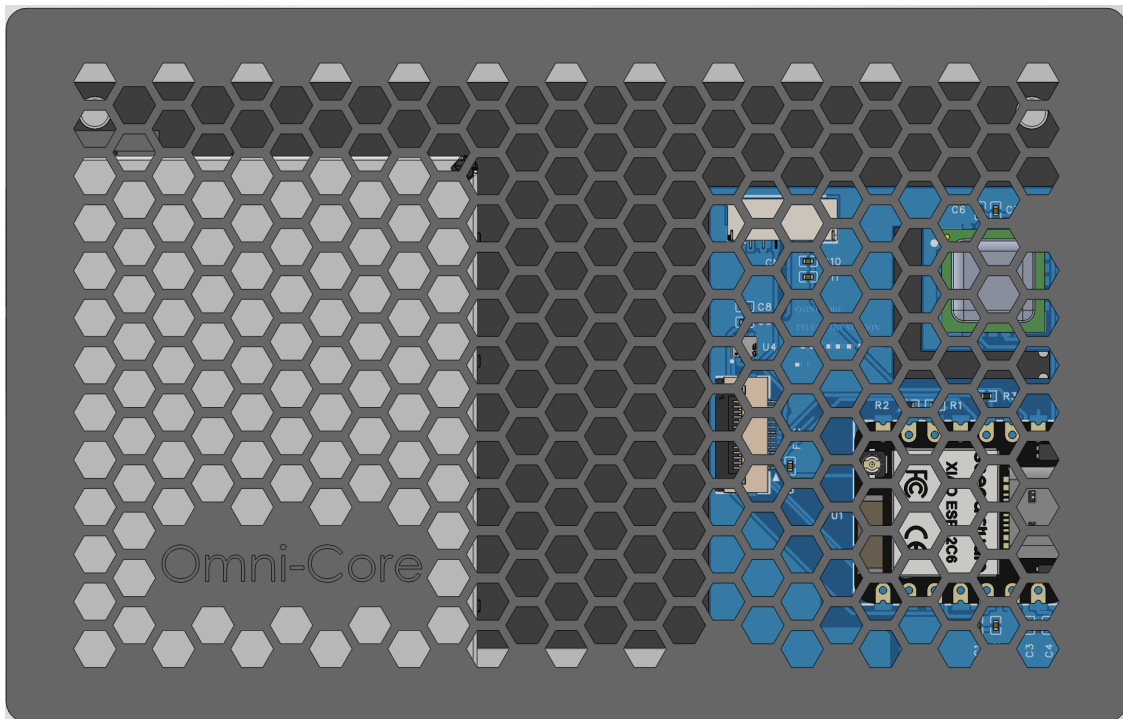


Figure 7: Back panel, straight-on view.

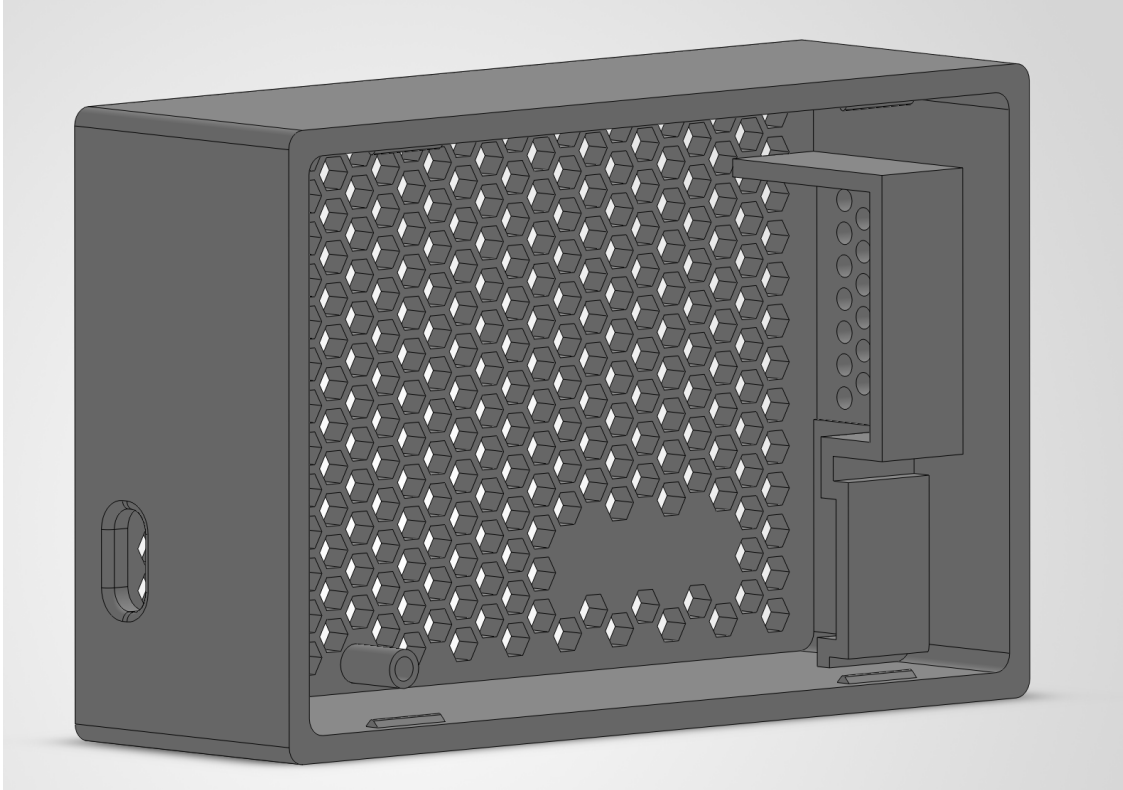


Figure 8: Enclosure body (lid removed), showing the SEN54 press-fit slot against the side vent.

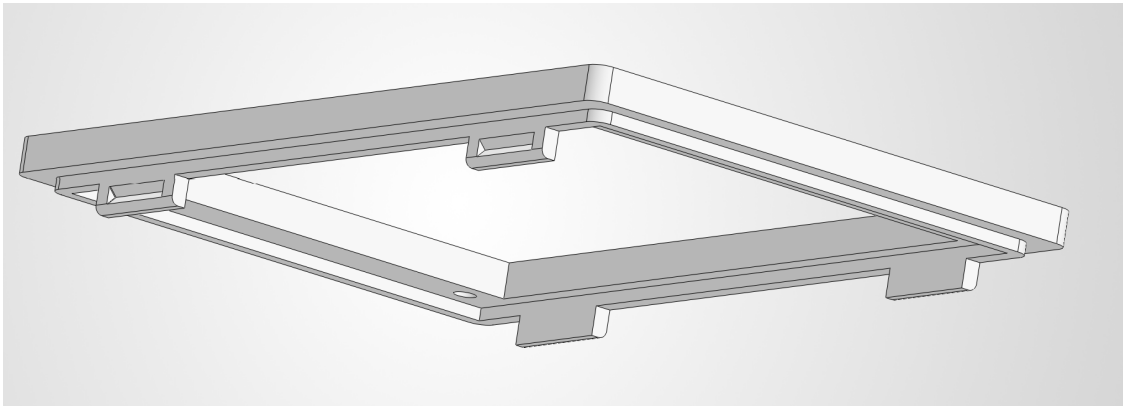


Figure 9: Lid piece, showing the display cutout and snap-fit tabs.

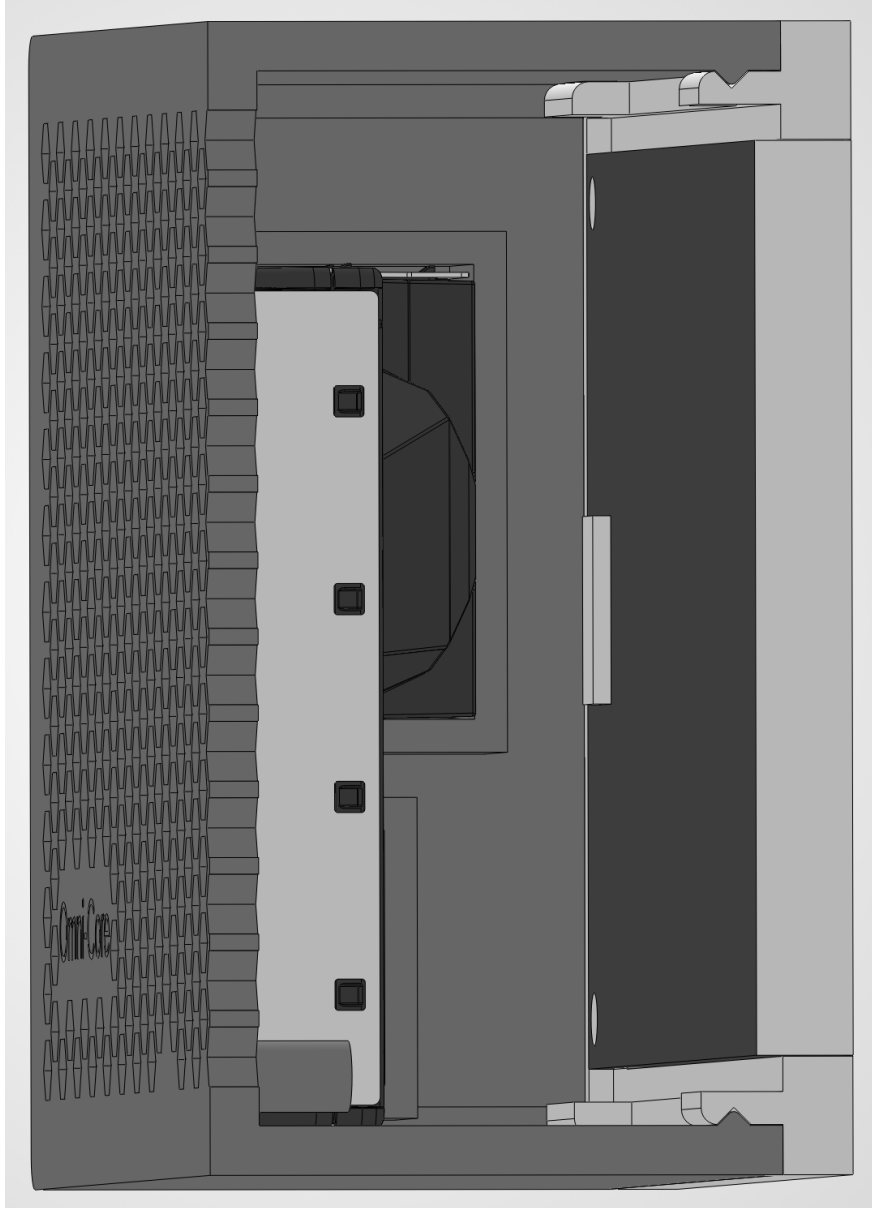


Figure 10: Cross-section through the lid/body joint, showing the snap-fit mating features.

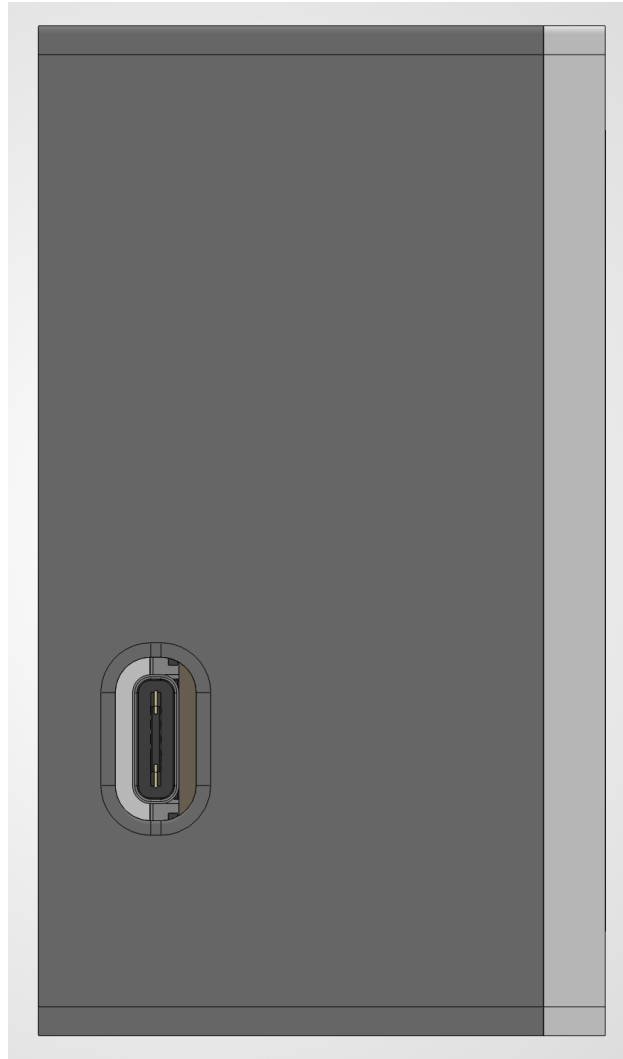


Figure 11: Side view, USB-C port cutout.

Understanding the Air Quality Data

The SCD40 and SEN54 report several different numbers, and it is easy to look at them without actually knowing what any of them mean or why the device bothers to color-code some of them. This section exists so that anyone reading the dashboard, not just someone who already knows air quality science, understands what they are looking at.

CO₂ – Carbon Dioxide

CO₂ is measured in parts per million (ppm). Outdoor air sits around 420 ppm. Indoors, in a room with people breathing and the windows closed, CO₂ climbs steadily, since every exhale adds more of it to the air and normal air exchange with the outside is often too slow to keep up.

CO₂ is not just a proxy for stuffiness. It has a direct, well documented effect on cognition.

Concentrations above roughly 800 ppm are associated with measurably reduced decision-making performance, and above 1200 ppm the effect becomes pronounced. This is why CO₂ is one of the two readings driving the dashboard’s overall air quality status: it is one of the few environmental numbers with a real, causal link to how sharp a person feels in a room. The fix is almost always simple. Crack a window or door and the number falls quickly.

Particulate Matter – PM1, PM2.5, PM4, and PM10

Particulate matter is solid or liquid material suspended in the air, measured in micrograms per cubic meter ($\mu\text{g}/\text{m}^3$). The number after “PM” refers to the size of particle being counted, in micrometers.

- **PM1** – the smallest particles the sensor tracks, roughly the size of smoke and combustion byproducts.
- **PM2.5** – particles 2.5 micrometers and smaller. This is the single most important particulate number, and the one most government air quality indexes are built around. Particles this small bypass the body’s natural filtering in the nose and throat and reach deep into the lungs, and can pass into the bloodstream. Wild-fire smoke, vehicle exhaust, and cooking smoke are all dominated by PM2.5-sized particles.
- **PM4** – a midpoint size class, less commonly referenced outside of raw sensor data, but useful as a transition point between fine and coarse particulates.
- **PM10** – particles up to 10 micrometers, which includes dust, pollen, and mold spores. These are still inhalable but are mostly filtered out before reaching deep into the lungs, so PM10 matters more for respiratory irritation than the deeper health risks associated with PM2.5.

The generally accepted safe range for PM2.5 is under 12 $\mu\text{g}/\text{m}^3$. Between 12 and 35 is considered elevated. Above 35 is considered unhealthy, particularly for sensitive groups. These are the exact thresholds the dashboard uses to color PM2.5 readings green, yellow, or red, both on the physical screen and on the web dashboard’s charts.

VOC Index

VOC stands for volatile organic compounds, a broad category covering off-gassing from things like cleaning products, paint, new furniture, and adhesives. The SEN54 does not report a raw concentration for any single compound. Instead, it runs an onboard algorithm that establishes a baseline for the air it has been breathing and reports an index, typically centered around 100, where higher numbers mean the air smells more different than usual to the sensor, not that a specific chemical has crossed a specific danger line. Because there is no single, widely agreed safe/unsafe threshold for this index the way there is for CO₂ and PM2.5, the dashboard displays it for awareness but does not use it to drive the overall air quality status.

Why CO2 and PM2.5 Specifically Drive the Status Indicator

Both the physical screen and the web dashboard show a single overall air quality verdict, GOOD, ELEVATED, or POOR, so that a glance at either one answers the only question that usually matters: is something worth doing about the air in this room right now. That verdict is computed from the worse of the CO2 and PM2.5 readings. Those two were chosen deliberately, because they are the two numbers on this device with real, specific, widely-agreed thresholds behind them, unlike VOC index, temperature, or humidity, which are either comfort metrics or lack an agreed-upon danger line.

Software Architecture

Firmware development was assisted by Claude Code.

Three-Layer Design

The codebase is organized into three distinct layers that never cross into each other's responsibilities.

Layer	Responsibility
Data Layer	Fetches data from the network or hardware sensors and writes results into shared state structs. Never touches the display.
Display Layer	Reads those structs and draws to the screen. Never makes network calls or reads hardware directly.
Coordinator	The main <code>loop()</code> function. Owns all timing decisions, gates network calls behind WiFi status checks, and decides when each layer runs.

This separation means that adding a new sensor is just filling in two functions. One reads the hardware and writes to shared state, and the other reads that state and draws to the screen. Nothing else in the codebase needs to change.

Struct-Scoped State

Earlier versions of this project kept every piece of runtime state as a flat global variable. As the project grew, this made it hard to tell at a glance which variables were related or which part of the code actually owned a given piece of state. Related state, sensor readings, touch input, persisted settings, and menu state, is now grouped into a small number of purpose-built structs instead, which makes the code considerably easier to navigate as the project keeps growing.

Non-Blocking Loop

The main loop is entirely non-blocking. Every repeating task is driven by a `millis()` interval check rather than a `delay()` call. This keeps the loop running continuously so touch input always feels instant and no task can block another.

```
1 // All timing decisions live in loop() -- nothing blocks
2 if (currentMillis - lastClockTime >= clockInterval) {
3     updateTimeDisplay();
4     lastClockTime = currentMillis;
5 }
```

Canvas Buffering

Several off-screen canvas buffers prevent flickering on every region of the display that updates on its own schedule. Without buffering, each drawing operation is a separate SPI transaction and the screen passes through partially drawn states that the eye sees as a flash. With a canvas, all drawing operations happen in RAM first and the finished frame is pushed to the display in a single atomic SPI transfer, so nothing ever appears half-drawn.

Color Palette System

All display colors are defined in a single `ColorPalette` struct returned by `getColorPalette()`. Night Mode was extended in v2.0 to cover the settings menu, brightness slider, and reset confirmation dialog in addition to the main dashboard, so the entire interface now shifts to the red palette together rather than leaving the menu at full brightness while the dashboard dims. Adding a new display element still just requires reading from the palette rather than hardcoding a color.

Settings Menu

Tapping anywhere on the dashboard opens a scrollable touch-driven settings menu. Current settings include Night Mode, temperature unit (F/C), military time (12/24hr), brightness, Auto Dim, and WiFi reset, with scroll arrows to page through the list and a live scrollbar showing the current position. Every tappable button on screen, the close button, the scroll arrows, and the reset confirmation's YES/NO buttons, shares its exact position between the code that draws it and the code that checks for a tap, so a button can never silently drift away from where it actually responds to a touch.

Settings Persistence

All user settings are saved to non-volatile storage immediately when changed and restored on every boot before the first frame draws. This means the device always starts up exactly as the user left it. Each setting is stored under a dedicated key in the "settings" NVS namespace, which is kept completely separate from the "wifi-gate" namespace used for WiFi credentials. A deep reset clears both namespaces simultaneously.

The Web Dashboard

Motivation

The physical display has always been, by design, a snapshot of right now. Adding a scrolling graph to a 480x320 panel that already shows a clock, weather, and nine sensor readings was never going to work without crowding everything else off the screen. Rather than compromise the physical dashboard, v2.0 adds a second, independent way to look at the same data: a web page served directly from the device itself, reachable at `http://omnicore.local` from any browser on the same network, with as much screen space as a laptop or phone can offer.

Live Data Endpoint

The device exposes a small JSON endpoint at `/data`, returning the current value of every live sensor reading. The web page polls this endpoint every two seconds and updates a grid of live cards, one per reading, each colored the same green, yellow, or red the physical screen would use for the readings that have defined thresholds.

History Ring Buffer

Alongside the live readings, the firmware keeps a 24-hour rolling history entirely in RAM. One sample is taken every 60 seconds and written into a fixed 1,440-slot circular buffer. Once the buffer fills, the next sample overwrites the oldest one automatically, since a circular buffer's write position simply wraps back to index zero once it passes the last slot. There is no separate “delete anything older than 24 hours” logic anywhere in the code; the 24-hour window falls directly out of the buffer's size (1,440 slots) multiplied by the sample interval (60 seconds), which is exactly 86,400 seconds, or 24 hours.

This buffer is intentionally RAM-only. The device runs continuously, so losing history on the rare reboot for a firmware update was judged an acceptable tradeoff against the real cost of writing to flash every minute for months, which would meaningfully wear down the flash chip over the device's lifetime.

A separate `/history` endpoint serves the entire buffer as a JSON array, oldest point first, every time it is called. Every history sample is timestamped using the device's NTP-synchronized wall clock rather than `millis()`, so that the browser and the device always agree on what a given point's real time actually was. An earlier version of this feature used boot-relative timestamps, which caused the browser's live-appended points and the device's historical points to disagree about what time it was, and produced a visibly broken, jumping timeline. Fixing this by having both sides agree on true epoch time resolved it completely.

Charts and Range Selection

The web dashboard renders five charts: CO2, particulate matter (all four PM size classes on one chart), VOC index, temperature, and humidity. The CO2 and PM2.5 lines are colored per-segment using the same thresholds as the live cards, so a line visibly shifts from green

to yellow to red as a room’s air quality changes over the day, rather than requiring a viewer to read the axis numbers to notice a problem.

A four-way range toggle (1 hour, 6 hours, 12 hours, 24 hours) lets a viewer zoom into recent fine detail or zoom out to the full day. The device is never asked for a different amount of data depending on which range is selected; the full 24-hour buffer is fetched once every 60 seconds and cached in the browser, and switching ranges simply re-slices that already-downloaded array to the most recent N points, which is instantaneous and requires no additional network request.

Air Quality Status Indicator

Both the web dashboard and the physical screen now display the same overall air quality verdict, computed identically in both places from the worse of the current CO2 and PM2.5 bands, as described in Section 4. Keeping this logic conceptually identical in both places, even though it is implemented once in C++ for the screen and once in JavaScript for the browser, was a deliberate choice, so that the two views of the device can never silently disagree about whether the room’s air is fine or not.

A Note on Multi-File Builds

An attempt was made during this phase of development to split the firmware’s single `main.cpp` into several separate `.cpp` files for readability. This was reverted. The specific combination of PlatformIO, the pioarduino platform, and this project’s pinned arduino-esp32 version produces an intermittent compiler error whenever more than one `.cpp` file includes `WiFi.h`, striking a different file on nearly every rebuild. After ruling out every source-level explanation, this was concluded to be a toolchain instability rather than a problem with the code itself; the single-file build has never failed under the same conditions. The one piece of that experiment kept is `webpage.h`, which holds the web dashboard’s HTML, CSS, and JavaScript in its own file that gets included back into `main.cpp`. A header include does not create a second compiled file the way a second `.cpp` does, so this keeps the readability benefit without triggering the build issue.

Reliability Design

Reliability was treated as a first-class requirement, not an afterthought. The following mechanisms work together to ensure the device recovers from any failure without manual intervention.

Mechanism	What it handles
Hardware watchdog (30s)	Resets the chip if <code>loop()</code> ever stalls, catching HTTP hangs, driver issues, or any other freeze
HTTP timeouts (5s response, 3s connect)	Prevents a slow or unresponsive API server from freezing the display
WiFi auto-reconnect (60s retry)	Quietly retries the connection in the background without disrupting the display
NTP resync debounce	Only resyncs time if it has been more than 60 seconds since the last sync, preventing hammering during a spotty connection
6-hour restart	If WiFi never comes back after 6 hours, the device restarts cleanly rather than sitting in a degraded state
Daylight saving detection	Resyncs location and time at 2:01 AM once per year to handle DST transitions automatically

The watchdog is intentionally armed after all blocking startup operations complete, to prevent false triggers during the WiFi connection loop on boot, which can take an unpredictable amount of time depending on network conditions. During this phase of development it was discovered that recent versions of the `arduino-esp32` framework auto-initialize the same watchdog timer before `setup()` ever runs, which silently caused this device's own 30-second timeout and panic-restart configuration to never actually take effect, since a second initialization on top of an already-running watchdog is rejected. The fix explicitly tears down whatever watchdog state already exists immediately before applying this device's own configuration, guaranteeing the intended settings are the ones actually running rather than losing quietly to whatever the framework initialized first.

Development History

The project was developed iteratively, with each version focused on a single clear goal.

v1.0 – Core Functionality

Established the foundation of Calloway OS. Implemented NTP-synchronized timekeeping and weather data from OpenWeatherMap. This version proved that the core concept, a glanceable dashboard driven entirely by internet data, was viable on the ESP32-C6 platform.

v1.1 – Network Intelligence

Replaced hardcoded WiFi credentials with a WiFiManager captive portal for first-time setup. Added automatic location detection via `ip-api.com`, which allowed timezone and weather city to be set dynamically from the network. The device became truly self-configuring.

v1.2 – Architecture Overhaul

Restructured the entire codebase around the three-layer architecture. Introduced non-blocking `millis()` intervals, separated data and display functions, and moved all timing decisions into `loop()`.

v1.3 – Reliability

Added the hardware watchdog, HTTP timeouts, WiFi health monitoring with auto-reconnect, the deep reset system, and the 6-hour restart fallback.

v1.4 – Hardware and Display

Brought the TSL2591 light sensor online. Added a TFT display with canvas-buffered rendering to prevent flickering. Implemented the Night Mode color palette system and the vertical lux bar on the display.

v1.5 – Settings and User Control

Added a scrollable settings menu with Night Mode, Celsius/Fahrenheit toggle, Military Time, Brightness control, Auto Dim, and WiFi reset with confirmation. All settings persist across reboots via NVS.

v1.6 – Local Sensor Integration

Brought a local temperature and humidity sensor online via the BME280, displayed in the top left of the dashboard. Added canvas buffering for the local sensor area and the date line, eliminating the last remaining sources of display flicker. A counterfeit SGP30 air quality sensor was also identified and discarded during this version, its serial ID memory was blank and its output values were impossible; a genuine replacement was ordered but a different sensor path was ultimately taken in v1.7. This LaTeX report itself was also started during this version.

v1.7 – Real CO2 and VOC

Added the SCD40 for genuine CO2, temperature, and humidity, and the SGP41 for VOC and NOx. This version also found and fixed a real I2C reliability issue: the TSL2591 was leaving the shared I2C bus in a state that broke communication with the SCD40 on a cold boot, fixed by explicitly resetting the bus between initializing the two sensors.

v1.8 – Larger Display

Upgraded from the 2.8" ILI9341 to the 3.5" ST7796 at 480x320, switching to the Arduino.GFX library in the process. The larger, higher-resolution screen is what made room for everything added in the versions since.

v1.9 – Touch Input

Replaced the hardware button entirely with an FT6236 capacitive touch controller. The settings menu was reworked around touch: scroll arrows, a brightness slider with live drag, and a two-step YES/NO confirmation before a WiFi reset is allowed to happen. The WiFi connection screen gained a hold-to-reset gesture as an escape hatch. The backlight also moved to direct PWM, since the new display has a built-in driver and no longer needs an external MOSFET. Auto Dim, mapping ambient light to brightness, was implemented here. Getting touch input reliable took some real debugging: a polling rate cap, a minimum tap duration, and a cooldown between taps were all needed to stop the touch controller from registering phantom touches.

v2.0 – Particulate Matter and the Web Dashboard

The SGP41 was replaced by the SEN54, adding real particulate matter readings (PM1, PM2.5, PM4, PM10) alongside its own VOC index. PM2.5 is now color-coded using the same EPA-style thresholds described in Section 4. This version also added the web dashboard: a live JSON data endpoint, a 24-hour in-RAM history buffer, and a browser-based set of charts with a 1/6/12/24-hour range toggle, described in full in Section 6.

Current Status and Future Work

What's Working Now

The device currently displays a live NTP-synchronized clock with 12 or 24-hour format, the current date, internet weather data including temperature, daily high and low, and sky conditions, local CO2, temperature, and humidity from the SCD40, full particulate matter and VOC readings from the SEN54, a live ambient light bar from the TSL2591, an overall air quality status indicator, a WiFi signal strength indicator, and a full touch-driven settings menu. All screen regions update without flickering thanks to canvas buffering throughout. Separately, a live web dashboard at `omnicore.local` mirrors the same data with a 24-hour history and interactive charts, viewable from any device on the same network.

Next Steps

- **PCB fabrication.** Moving from the current prototype to a custom PCB. See Section 3.6 for details once this is finalized.
- **3D printed enclosure.** Designing a proper enclosure around the finished PCB and display. See Section 3.7 for details once this is finalized.
- **Matter integration.** Investigating whether the SEN54's particulate readings can be exposed as a Matter endpoint, pending either upstream support from Espressif's Arduino core or a hand-written endpoint wrapper following the pattern used by existing community air-quality Matter implementations.

- **Sleep timeout.** An additional menu setting to dim or turn off the display after a configurable period of inactivity.

Conclusion

The Omni-Core Telemetry Station started as a desk clock and grew into something more interesting. It is a small device that knows where it is, keeps itself alive, and shows useful real-world data without being told what to do, and it now does that in two places at once: a glanceable physical screen for right now, and a web dashboard for everything that led up to it. The engineering philosophy that shaped it, thinking about failure modes before writing code, keeping functions honest about what they do, and making the architecture easy to extend, is the same philosophy that shows up in professional embedded systems at every scale, and the same philosophy that made it possible to add an entire second interface to this device without touching the reliability guarantees the first one was built on.

The hardware is stable and both environmental sensors, plus touch input, are fully online. The software has been reorganized around struct-scoped state rather than flat globals, and two real bugs (a silently-inactive watchdog configuration, and a history timeline that desynced over long sessions) were found and fixed along the way. What remains is the physical side: a finished PCB and an enclosure to put it in.

Appendix: Pinout and Wiring Reference

This appendix collects the complete electrical reference for the build: every pin assignment on the XIAO ESP32-C6, the shared I2C bus and device addresses, and the supporting power/decoupling network, as built on the current PCB revision described in Section 3.6.

XIAO ESP32-C6 Pin Assignments

Pin	Net	Function
D0	LCD_RST	Display reset
D1	LCD_CS	Display chip select
D2	PWM	Backlight, direct PWM drive to display LED pin
D3	LCD_RS	Display data/command (DC)
D4	SDA	I2C data – shared by all sensors and touch
D5	SCL	I2C clock – shared by all sensors and touch
D6	–	Unused
D7	CTP_RST	Touch controller hardware reset
D8	SCK	Display SPI clock
D9	CTP_INT	Touch controller interrupt (wired, not currently used in firmware)
D10	MOSI	Display SPI data

The display is write-only over SPI; no MISO connection is present.

Shared I2C Bus

All I2C devices, the touch controller and all three environmental sensors, share a single bus on D4 (SDA) and D5 (SCL), running at 400kHz and pulled up to 3.3V with 4.7k Ω resistors on both lines.

Device	Function	I2C Address
FT6236	Capacitive touch controller	0x38
SCD40	CO2 / temperature / humidity	0x62
SEN54	Particulate matter (PM1/2.5/4/10) + VOC index	0x69
TSL2591	Ambient light	0x29

Power and Decoupling

- Bulk 10 μ F capacitors sit on both main supply rails, one on 3.3V and one on 5V.
- 1 μ F capacitors decouple the SCD40 and the TSL2591 individually.
- A 10 μ F bulk capacitor decouples the SEN54's supply pins, sized larger than the other sensors to cover its internal fan's current draw.
- 100nF ceramic capacitors are placed at the SCD40, TSL2591, display FPC connector, and SEN54 supply pins, as close as possible to each device's power pin, to keep

high-frequency switching noise from coupling into the shared I2C bus.

- I2C pull-ups: $4.7k\Omega$ on both SDA and SCL to 3.3V.

Datasheet Links

Component	Source
XIAO ESP32-C6	Seeed Studio Wiki
ST7796 3.5" display module	LCD Wiki (manufacturer)
FT6236 touch controller	Adafruit
TSL2591 light sensor	ams (via Adafruit)
SCD40 CO2 sensor	Sensirion
SEN54 particulate sensor	Sensirion

Bill of Materials

Designator	Component	Part Number
U1	XIAO ESP32-C6	1597-113991254-ND
U2	SCD40-D-R2	1649-SCD40-D-R2CT-ND
U4	TSL25911FN	TSL25911FNCT-ND
CN1	SM06B-GHS-TB (SEN54 header)	455-1568-1-ND
FPC	FFC2B35-14-G (display connector)	2073-FFC2B35-14-GCT-ND
C2, C4	10 μ F 0402 cap	490-GRM155R60J106ME05DCT-ND
C5, C7	1 μ F 0402 cap	1276-1067-1-ND
C1, C3, C6, C8-C10	100nF 0402 cap	1276-CL05B104KP5VPNCCT-ND
R1, R2	4.7k Ω 0402 resistor	311-4.7KJRCT-ND
R3	100 Ω 0402 resistor	541-3239-1-ND
–	SEN54 (particulate/VOC sensor)	SEN54-SDN-T

Figures

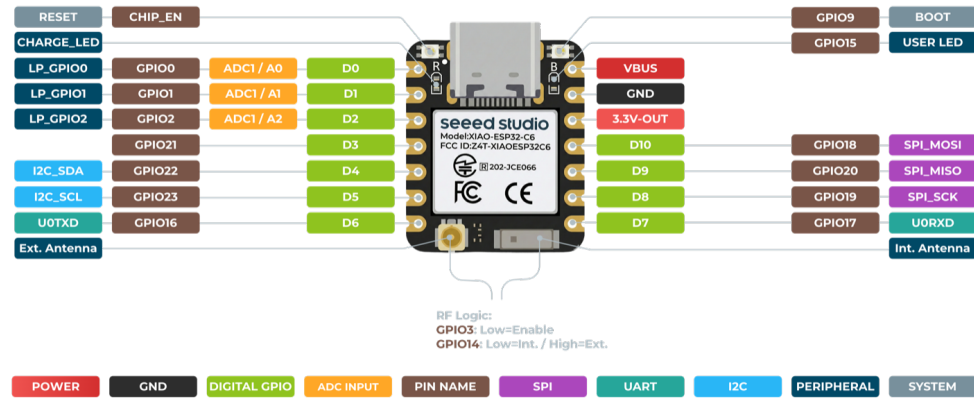


Figure 12: XIAO ESP32-C6 pinout, front (Sseed Studio).